

Incident INC-006 [P1 - Kubernetes]: Production Pod CrashLoopBackOff - OOMKilled. Description: All 6 replicas of deployment 'pricing-service' in namespace commerce entered CrashLoopBackOff at 18:50 UTC, ~4 minutes after rollout of image v2.4.1. Pods exited with code 137 (OOMKilled) after ~45s. Pricing API returned 503 for 100% of traffic for 11 minutes; downstream cart and checkout degraded simultaneously. Detected by Prometheus alert KubePodCrashLooping firing on all pods.. Triage Steps: 1) Confirm the symptom: `kubectl get pods -n commerce -l app=pricing-service -o wide` 2) Describe a failing pod and capture exit code: `kubectl describe pod pricing-service-7d8c9-xyz -n commerce | grep -A 5 'Last State'` 3) Read previous-container logs: `kubectl logs pricing-service-7d8c9-xyz -n commerce --previous --tail=200` 4) Verify reason is OOMKilled: `kubectl get pod pricing-service-7d8c9-xyz -n commerce -o jsonpath='{.status.containerStatuses[0].lastState.terminated.reason}'` 5) Compare memory limit vs JVM Xmx: `kubectl get deployment pricing-service -n commerce -o jsonpath='{.spec.template.spec.containers[0].resources}'` 6) Inspect JVM flags inside the image: `kubectl debug -it pricing-service-7d8c9-xyz -n commerce --image=busybox -- cat /proc/1/cmdline | tr '\0' ' ' 7) Diff against previous image manifest: docker image inspect registry.internal/pricing-service:v2.4.0 v2.4.1 | jq '[]|.Config.Env' 8) Check recent code changes: git log v2.4.0..v2.4.1 --oneline --stat services/pricing/. Root Cause: Image v2.4.1 bumped the base image from eclipse-temurin:17-jre-alpine to eclipse-temurin:21-jre-alpine, which changed the JVM's default MaxRAMPercentage behaviour. Combined with an unbounded -Xmx and a 1Gi container limit, the JVM tried to allocate ~900Mi of heap and overflowed the cgroup memory ceiling within the first warm-up cache load.. Resolution: 1) Immediately roll back: kubectl rollout undo deployment/pricing-service -n commerce 2) Verify rollout: kubectl rollout status deployment/pricing-service -n commerce --timeout=120s 3) Patch the deployment to set memory request=768Mi, limit=1536Mi: kubectl set resources deployment pricing-service -n commerce --requests=memory=768Mi --limits=memory=1536Mi 4) Update Dockerfile to set explicit JVM heap: ENV JAVA_TOOL_OPTIONS='-XX:MaxRAMPercentage=70 -XX:InitialRAMPercentage=70 -Xss512k' 5) Build v2.4.2, run JVM warm-up load test locally with the new memory ceiling for 10 minutes 6) Deploy v2.4.2 to canary (1 pod) and monitor JVM metrics (jvm_memory_used_bytes) for 30 minutes 7) Roll out to full fleet once canary is steady and no OOMKilled events observed 8) Add Prometheus alert: container_memory_working_set_bytes / container_spec_memory_limit_bytes > 0.85 for 5m. MTTR: 17 minutes. Lessons Learned: JVM containers must declare explicit -XX:MaxRAMPercentage or -Xmx that fits inside the cgroup limit with at least 25% headroom for metaspace, code cache and native buffers. Base-image bumps require a heap-sizing review..`